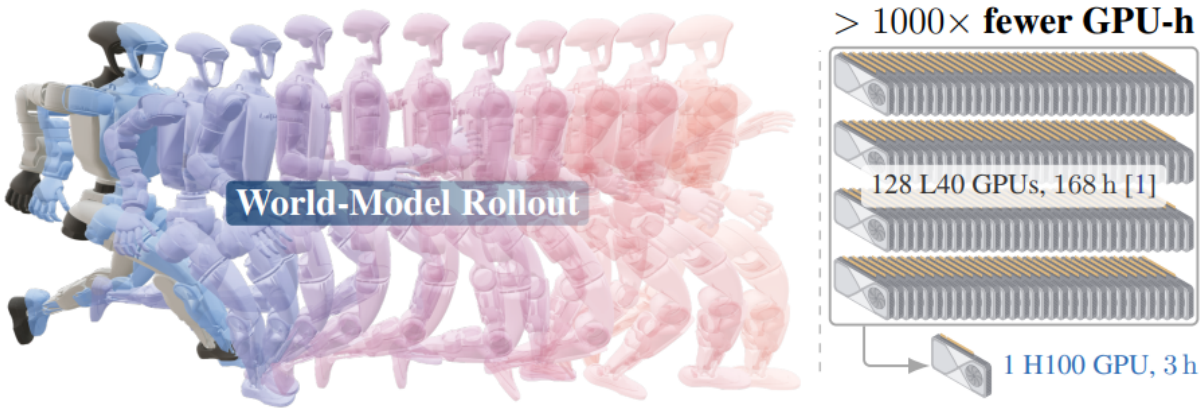
SuperMimic Reading Notes

Humanoid Control, Policy Gradients,
and World Models

PPO / BPTT / DIFFERENTIABLE MBRL

hikaru



October 2026

Contents

Before We Begin . . . 2

1 Two Routes to Policy Training **2**

2 MFRL **4**

2.1 Cumulative Return and the Policy Objective . . . 4
2.2 Policy Gradients and Advantage . . . 4
2.3 TRPO and PPO . . . 5
2.4 Training Properties of Policy Gradients . . . 6

3 MBRL **7**

3.1 Dynamics Models and Policy Optimization . . . 7
3.2 Closed-Loop BPTT . . . 8
3.3 Physical Delay and Credit Assignment . . . 9

4 Analyzing BPTT **11**

4.1 Model Error and Derivative Error . . . 11
4.2 Long-Horizon Jacobians and Gradient Propagation . . . 13
4.3 Contact Thresholds and Gradient Estimation . . . 13
4.4 Data Coverage and Exploration . . . 15

5 SuperMimic **15**

5.1 Alternating Training and Experience Replay . . . 16
5.2 World Models and Multi-Step Supervision . . . 17
5.3 Short Rollouts and Terminal Value . . . 18
5.4 Termination Risk and Survival Weights . . . 19
5.5 Critics and Multi-Step Returns . . . 19
5.6 Data Sampling and Deployment Interfaces . . . 20

6 Results **21**

6.1 Training Efficiency and Motion Scale . . . 21
6.2 Horizon Ablations and Push Recovery . . . 21
6.3 Real G1 Control . . . 22

7 Reproduction **22**

8 Action World and WAM **23**

A Notation and Training Interfaces **24**

References **25**

Before We Begin

SuperMimic is work by Qiayuan Liao and colleagues at UC Berkeley, published at CoRL 2026. [1]

Humanoid whole-body control aims to make the same robot reliably walk, run, jump, roll, and perform other movements. Human motion offers a rich collection of behavior examples, while the robot must adjust its joint commands to its actual posture, velocity, and contacts. Turning motion data into a feedback controller that runs on a real body is the common problem behind these systems.

For this problem, model-free RL (MFRL), represented by SONIC, has developed into a mature approach: collect samples in large-scale simulation and train a policy with motion-tracking rewards. [8] Model-based RL (MBRL) instead learns how the body changes after an action and uses that model to optimize the policy. The dynamics of the same body can be reused to train different motions; the model can also provide local gradients of how actions affect the future. This makes MBRL attractive for training efficiency, while introducing difficulties involving model error, contact, and differentiation over long horizons.

SuperMimic follows this approach, combining shared dynamics, short-horizon optimization, and long-term value estimation for large-scale humanoid control. Understanding it offers a chance to revisit why MBRL is natural, why it is difficult, and how to make it work well for this task. To borrow a joke: *Make MBRL Great Again*.

These notes start with the technical foundations of humanoid control and derive PPO and BPTT from the same long-term return objective, examining their optimization properties and the difficulties MBRL encounters. They then return to SuperMimic to study model fitting, short-horizon actor–critic training, data coverage, and deployment interfaces, before examining training efficiency and recovery behavior through the paper's experiments. The final discussion connects the body-dynamics model here to models of the external world in robot manipulation.

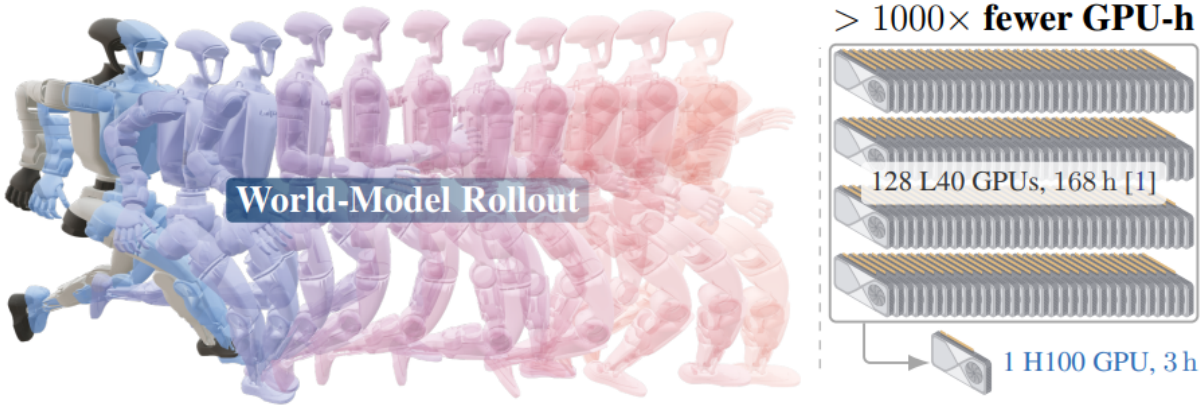


Figure 1 from the SuperMimic paper. A shared differentiable world model trains a policy for multiple whole-body skills. [1]

1 Two Routes to Policy Training

There are currently two main routes to training policies for humanoid whole-body motion. Model-free RL (MFRL) collects trajectories in a physics simulator and updates the policy from returns, most commonly with PPO. Model-based RL (MBRL) learns the robot's dynamics, uses that model to predict motion, and optimizes the policy by backpropagating through the dynamics. Both address the same problem: turning human motion data into a feedback controller that can execute on a real robot.

Progress on this task has depended on motion data, physics simulation, and real-robot deployment. DeepMimic demonstrated early on how motion capture and reinforcement learning could train imitation policies. [2] H2O later addressed human-to-robot motion retargeting and feasibility filtering; ASAP used real-robot data to correct discrepancies in simulated dynamics; Hold My Beer studied coordination between upper- and lower-body objectives and control rates; and BeyondMimic further improved tracking and deployment of diverse dynamic motions. [4], [5], [6], [7] Meanwhile, GPU-parallel simulation greatly accelerated PPO training. [3] Together, these advances made large motion libraries increasingly learnable and executable on real hardware.

SONIC marks an important turning point in this development. By scaling motion data, network capacity, and training compute together, it trains a single general-purpose tracker covering a broad range of movements. [8] This shows that MFRL, particularly PPO, can handle large-scale humanoid motion tracking and has become a highly competitive mainstream approach. That scale still carries a substantial training cost. The released SONIC model compared in SuperMimic used 128 L40 GPUs for 168 hours, or about 21,500 GPU-hours. [1] As motion libraries grow to hundreds of hours, improving the computational efficiency of policy training deserves renewed attention.

The MBRL route has also continued to develop. As early as 2021, SuperTrack learned differentiable dynamics from simulator transitions and trained a motion tracker through model rollouts and backpropagation. [9] SuperDyno later extended this idea to larger motion libraries. [10] Whereas PPO mainly estimates policy gradients from sampled returns, these systems seek to use the dynamics information in each transition to calculate directly how changes in an action affect future return.

SuperMimic continues this route by combining a shared differentiable dynamics model, short-horizon policy optimization, and a value function for large-scale G1 whole-body motion tracking. [1] A key to understanding SuperMimic is therefore to understand how PPO and differentiable dynamics use the same motion trajectory, and the strengths and difficulties of their respective gradient updates. We begin with a common cumulative-return objective and derive PPO and BPTT in turn.

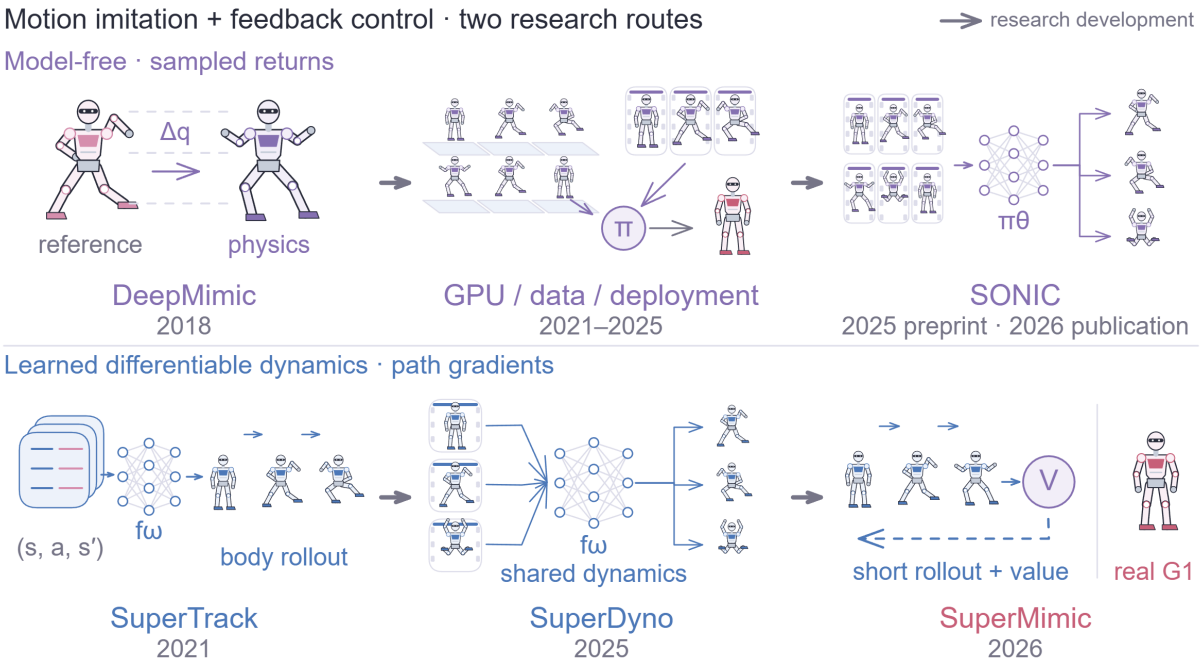


Figure 1. Two research routes for motion imitation. Thick gray arrows connect stages of development; thin colored arrows show local data and prediction relationships. The upper route learns control from sampled returns, while the lower route learns differentiable dynamics and uses pathwise gradients.

2 MFRL

The motion-tracking task ultimately needs to turn reference motion, balance, and recovery requirements into training signals. MFRL has the robot try actions in a simulator and adjusts its policy using returns over complete movements. Taking PPO as an example, we start with its objective, examine how each action receives a long-term evaluation, and see why this statistical route suits large-scale simulation training.

2.1 Cumulative Return and the Policy Objective

Tracking error can contribute to reward at each step, while the consequences of landing or falling may appear many steps later. Combining these evaluations makes cumulative return over the motion the policy's objective. Let s_t be the environment state, a_t the action, $r_{t+1} = r(s_t, a_t, s_{t+1})$ the reward after executing it, and $\gamma \in [0, 1)$ the discount factor. The policy parameters are θ and the environment transition is P. Here, the state includes goal information and the history needed to predict what follows; for a finite horizon, it also includes time. The policy seeks to maximize

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[R(\tau)], \qquad R(\tau) = \sum_{t=0}^{T-1} \gamma^t r_{t+1}. \tag{2.1}$$

The probability of a trajectory $\tau = (s_0, a_0, s_1, \ldots, s_T)$ factorizes as

$$p_\theta(\tau) = p_0(s_0) \prod_{t=0}^{T-1} \underbrace{\pi_\theta(a_t \mid s_t)}_{\text{policy selects an action}} \underbrace{P(s_{t+1} \mid s_t, a_t)}_{\text{environment produces an outcome}}.$$

Here, P can come from a rigid-body physics simulator. Policy parameters affect the probability of sampling actions and, through those actions, the distribution of subsequent states.

The policy decides which actions to try, and the simulator supplies their consequences. For a Gaussian policy, common in continuous control, the network outputs $\mu_\theta(s)$ and $\sigma_\theta(s)$, which define $\pi_\theta(a \mid s)$; an action is then sampled from this distribution. Each execution produces a trajectory and a cumulative return, allowing the objective to be estimated by averaging interactions. For N independent trajectories, the empirical mean

$$\widehat{J}_N = \frac{1}{N} \sum_{i=1}^{N} R(\tau^{(i)})$$

converges to J by the law of large numbers when return is integrable; with finite variance, its variance is $\mathrm{Var}(R)/N$. The policy network defines a distribution over actions, and sampling lets us estimate the actual return that distribution produces in the environment.

2.2 Policy Gradients and Advantage

The next step is to turn return samples into a direction for updating parameters. Policy gradients work through trajectory probabilities: actions in high-return trajectories should become more likely, and actions in low-return trajectories less likely. Differentiating the objective using $\nabla p = p \nabla \log p$ gives

$$\nabla_\theta J = \int R(\tau)\, \nabla_\theta p_\theta(\tau)\, d\tau \qquad (2.2)$$

$$= \mathbb{E}_{p_\theta}\left[R(\tau) \nabla_\theta \log p_\theta(\tau)\right] \qquad (2.3)$$

$$= \mathbb{E}_{p_\theta}\left[R(\tau) \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]. \qquad (2.4)$$

The last step follows because environment transitions have no explicit dependence on θ: only policy terms in the trajectory log probability need differentiation. Collision, friction, and landing already affect the sampled return; optimization computes how the policy network changes the probabilities of those actions.

A full trajectory return mixes outcomes before and after action a_t. Since this action can affect only later rewards, separating rewards by time makes the expected product of past return and the current score term zero. This yields the reward-to-go form

$$\nabla_\theta J = \mathbb{E} \sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t, \quad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_{k+1}. \qquad (2.5)$$

The conditional expectation of G_t is the action value $Q^\pi(s_t, a_t)$. However, the current state alone may make returns high or low for all actions. To evaluate the improvement due to the current action more clearly, we can subtract a state-only baseline $b(s_t)$. This leaves the expected gradient unchanged because

$$\mathbb{E}_{a\sim\pi(\cdot|s)}[b(s)\nabla_\theta \log \pi_\theta(a \mid s)] = b(s)\nabla_\theta \int \pi_\theta(a \mid s)\, da = 0.$$

The state value $V^\pi(s_t)$ is the mean cumulative return from the current state under the current policy. Subtracting it evaluates how much better the selected action performs than the usual continuation. Taking $b = V^\pi$ gives the advantage $A^\pi = Q^\pi - V^\pi$ and the policy gradient

$$\nabla_\theta J = \mathbb{E} \sum_t \gamma^t \boxed{\nabla_\theta \log \pi_\theta(a_t \mid s_t)} \boxed{A^\pi(s_t, a_t)}. \qquad (2.6)$$

The **blue term** gives the parameter direction that increases the probability of the sampled action. The sign of the **purple advantage** determines whether to update along or against that direction, and its magnitude weights the update. This evaluation can include landing, recovery, or task success far in the future.

For example, if a landing reward appears two seconds after takeoff, G_t still brings it back to the takeoff action. Rewards cross time through return and advantage, while the parameter gradient itself backpropagates through the policy network. This structure explains how model-free policy gradients use long-term outcomes. [11], [12]

2.3 TRPO and PPO

The gradient above is computed under the current policy's trajectory distribution. In practice, we want one sample batch to support multiple updates, but the policy changes after the first one. Reusing the batch requires describing the difference between the probabilities the old and new policies assign to the same action. Write the data-collection policy as π_{old} and the new policy as π_θ. The one-step probability ratio is

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}.$$

At the same state, importance sampling rewrites an expectation under the new action distribution as a weighted expectation under the old one:

$$\mathbb{E}_{a\sim\pi_\theta}[A_{\text{old}}(s,a)] = \mathbb{E}_{a\sim\pi_{\text{old}}}[\rho(\theta)A_{\text{old}}(s,a)].$$

The one-step ratio accounts for a change in action distribution at a fixed state. Continuous control must also consider which states the new policy will visit. For an infinite horizon, let $d^\pi(s) = (1-\gamma)\sum_{t\geq 0}\gamma^t P(s_t = s \mid \pi)$. The performance difference identity gives

$$J(\theta) - J(\text{old}) = \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi_\theta},\, a\sim\pi_\theta}[A_{\text{old}}(s,a)].$$

Approximating the updated state distribution by $d^{\pi_{\text{old}}}$ yields a local surrogate. TRPO optimizes it while constraining the mean KL distance between the old and new policies:

$$\begin{aligned} \max_\theta \quad & \mathbb{E}_{\text{old}}[\rho_t(\theta)\widehat{A}_t], \\ \text{s.t.} \quad & \mathbb{E}_{s\sim d^{\pi_{\text{old}}}}\left[D_{\text{KL}}(\pi_{\text{old}}(\cdot \mid s)\|\pi_\theta(\cdot \mid s))\right] \leq \delta. \end{aligned} \tag{2.7}$$

Constraining the update size makes old data more useful for evaluating the new policy. PPO expresses this control through a clipped objective that is convenient for minibatch optimization:

$$L^{\text{clip}}(\theta) = \mathbb{E}_{\text{old}}\left[\min\left(\rho_t\widehat{A}_t, \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\widehat{A}_t\right)\right]. \tag{2.8}$$

When $\widehat{A}_t > 0$, the sample favors increasing the action probability; once $\rho_t > 1+\varepsilon$, a further increase no longer raises its clipped benefit. When $\widehat{A}_t < 0$, it favors decreasing the probability; once $\rho_t < 1-\varepsilon$, that benefit also becomes flat. This weakens the incentive to overoptimize old samples and allows multiple minibatch updates. Clipping acts on the optimization benefit. [12]

The same trajectory batch thus supplies long-term evaluations for individual actions and supports multiple rounds of minibatch optimization; the clipped objective controls the update size when repeatedly reusing old samples.

2.4 Training Properties of Policy Gradients

Equation (2.6) separates the update into a direction of probability change and a return-based evaluation. Physical consequences enter through sampled return, so contact switches and termination thresholds can directly affect action probabilities. A stochastic policy also samples beyond its current mean. If a useful behavior occurs with probability p per attempt, the probability of observing it at least once in N independent attempts is $1-(1-p)^N$.

This route also makes good use of GPU simulation: environments collect trajectories in parallel, and updates estimate a direction from many return samples. Baselines, a state-value network (critic), and advantage estimation help reduce statistical fluctuations. Long-term outcomes enter through advantage, and policy backpropagation need not pass through the entire physical process.

The discussion above gives three main properties of PPO:

1. **Parallel efficiency.** Multiple simulation environments collect trajectories simultaneously, and policy updates do not backpropagate through the full physical dynamics, making the method suitable for large-scale GPU simulation.
2. **Training stability.** Baselines and critics help reduce gradient-estimation fluctuations, while clipping discourages overoptimization of old samples and supports relatively stable policy iteration.
3. **Exploration and long-term evaluation.** Stochastic policies try actions beyond their current mean, and advantage brings later returns into the evaluation of the current action.

At the same time, each direction must still be identified through action trials and differences in return. High-dimensional actions vary in many ways, while distant rewards mix the effects of intervening steps. MBRL tries to use the body's shared dynamics in the simulator to calculate how action changes affect return, obtaining a more specific optimization direction from each trajectory.

3 MBRL

PPO repeatedly tries actions and estimates which changes accompany higher return. If we can predict how the body moves after an action and differentiate that prediction, we can calculate an improvement direction directly through the relationships among actions, states, and rewards. This is the starting point for the training efficiency of differentiable MBRL: reuse collected transition experience through a model, then let one rollout provide local directional information.

3.1 Dynamics Models and Policy Optimization

A differentiable path first requires a model that predicts the next state. Consider a deterministic controller $a_t = \mu_\theta(s_t)$. Collect states, actions, and next states from the simulator to learn

$$\hat{s}_{t+1} = f_\omega(\hat{s}_t, a_t),$$

where ω denotes dynamics-model parameters and θ policy parameters. Model training changes ω to match the simulator; policy training temporarily fixes ω and increases model-based return. The two optimization problems use the same interaction experience but have distinct objectives and differentiation variables.

After fitting the model, we can test the controller inside it. Let H be the explicit rollout length for this update. Starting from a recorded state $\hat{s}_0 = s_0$, model and policy alternate:

$$\hat{s}_0 \xrightarrow{\mu_\theta} a_0 \xrightarrow{f_\omega} \hat{s}_1 \xrightarrow{\mu_\theta} a_1 \xrightarrow{f_\omega} \cdots \xrightarrow{f_\omega} \hat{s}_H.$$

Initially, summing only these H model rewards gives an objective within the window. When H is shorter than the full motion, returns beyond the window need a separate continuation. The imagined objective within the current window is

$$\hat{J}_H(\theta) = \mathbb{E}_{s_0}\left[\sum_{t=0}^{H-1} \gamma^t \hat{r}_{t+1}\right], \qquad \hat{r}_{t+1} = r_\omega(\hat{s}_t, a_t). \tag{3.1}$$

Both PPO and model-based policy optimization pursue cumulative motion return; the model objective written here computes the part within the explicit window. Policy parameters affect later states through each action. Applying the chain rule to this graph unrolled in time gives backpropagation through time, or BPTT. State predictions keep the rollout going, while local Jacobians tell the policy how to change its actions. SVG, SuperTrack, and related work use such differentiable paths. [14], [9]

3.2 Closed-Loop BPTT

To see how a parameter change reaches the future, first track the total derivative of each state with respect to policy parameters. Let the dimensions of states, actions, and parameters be n, m, p, respectively. Define

$$D_t = \frac{d\hat{s}_t}{d\theta} \in \mathbb{R}^{n\times p}, \quad f_{s,t} = \frac{\partial f_\omega}{\partial s} \in \mathbb{R}^{n\times n}, \quad f_{a,t} = \frac{\partial f_\omega}{\partial a} \in \mathbb{R}^{n\times m},$$

$$\mu_{s,t} = \frac{\partial \mu_\theta}{\partial s} \in \mathbb{R}^{m\times n}, \qquad \mu_{\theta,t} = \frac{\partial \mu_\theta}{\partial \theta} \in \mathbb{R}^{m\times p}.$$

All partial derivatives are evaluated at the current trajectory's states and actions. In this section, a scalar function's derivative with respect to a vector is written as a row Jacobian; its transpose gives a column parameter gradient. Policy parameters change a_t through two paths: directly changing policy output, and changing an earlier state that then changes the action through policy feedback. Thus

$$\frac{da_t}{d\theta} = \mu_{s,t} D_t + \mu_{\theta,t}.$$

Substituting into the derivative of the state transition gives

$$\begin{aligned} D_{t+1} &= f_{s,t} D_t + f_{a,t} \frac{da_t}{d\theta} \\ &= \boxed{(f_{s,t} + f_{a,t}\mu_{s,t})} D_t + \boxed{f_{a,t}\mu_{\theta,t}}. \end{aligned} \tag{3.2}$$

Write $F_t = f_{s,t} + f_{a,t}\mu_{s,t}$ and $B_t = f_{a,t}\mu_{\theta,t}$. The closed-loop state propagation is F_t: f_s describes direct propagation of the state, while $f_a\mu_s$ describes propagation through the controller's response and its action. The term B_t is the new influence injected into the dynamics by policy parameters at this step.

The data starting point is fixed with respect to the parameters for this update, so $D_0 = 0$. Repeatedly unrolling the recurrence gives

$$D_T = \sum_{k=0}^{T-1} \underbrace{F_{T-1} F_{T-2} \cdots F_{k+1}}_{\text{step } k \text{ influence propagates through feedback}} B_k, \tag{3.3}$$

An empty product is the identity matrix. For $\hat{r}_{t+1} = r_\omega(\hat{s}_t, a_t)$, its derivative is

$$\frac{d\hat{r}_{t+1}}{d\theta} = (r_{s,t} + r_{a,t}\mu_{s,t}) D_t + r_{a,t}\mu_{\theta,t}.$$

The gradient $\nabla_\theta \hat{J}_H$ is the discounted sum of these reward derivatives. Automatic differentiation usually propagates vector products in reverse order; the forward sensitivity recurrence above makes the same dependencies easier to see.

If we isolate the effect of a single action intervention a_0 on the final state while allowing subsequent control feedback to continue, then

$$\frac{\partial \hat{s}_T}{\partial a_0} = \boxed{F_{T-1} \cdots F_1} f_{a,0}. \tag{3.4}$$

This equation writes how takeoff affects landing as a product of local effects. Each step rescales and rotates the perturbation. The long-horizon propagation problem discussed later occurs directly in this term.

These local derivatives also change the optimization information available from one trajectory. Equation (2.6) adjusts the probability of a sampled action according to advantage, while BPTT calculates the return change caused by an action change. Reduce the problem to one scalar action: fix its standard deviation $\sigma > 0$, let policy parameter θ control its mean, and take a linear return:

$$a = \theta + \sigma z, \qquad z \sim \mathcal{N}(0,1), \qquad R(a) = c + ba.$$

Use $V = \mathbb{E}[R(a)] = c + b\theta$ as the baseline, and suppose the model has accurately learned slope b. At the start of a PPO update, the score estimator and pathwise derivative give

$$g_{\text{score}} = (R(a) - V)\nabla_\theta \log \pi_\theta(a) = (b\sigma z)\frac{z}{\sigma} = bz^2,$$
$$g_{\text{path}} = \frac{\partial R}{\partial a}\frac{\partial a}{\partial \theta} = b.$$

Using $\mathbb{E}[z^2] = 1$ and $\mathbb{E}[z^4] = 3$ yields

$$\mathbb{E}[g_{\text{score}}] = \mathbb{E}[g_{\text{path}}] = b,$$
$$\text{Var}(g_{\text{score}}) = 2b^2, \qquad \text{Var}(g_{\text{path}}) = 0.$$

The estimators have the same mean direction. Averaging N independent score samples reduces their variance to $2b^2/N$, whereas the pathwise derivative in this linear example gives direction b in one evaluation. The difference comes from the model retaining the local relationship between action and outcome, which differentiation can use directly. In a multi-step system, BPTT connects these local relationships over time, so one rollout provides directions for how each action affects later return. [14] Transition and derivative errors also enter the update along this path; the next chapter examines them.

Fitting dynamics costs extra computation. Given state, action, and physical conditions, however, the transition law does not change with the collection policy, so past experience remains useful for supervision. The same body's dynamics can also be reused across motions and policy updates, amortizing fitting cost over later training. Generating transitions inside the model reduces simulator calls, while pathwise differentiation can reduce the rollouts needed to estimate an update direction. Computation per step and the number of steps needed to reach a target return correspond to throughput and sample efficiency, respectively; Figure 7 compares both. [1]

3.3 Physical Delay and Credit Assignment

The product above is necessary because an action often changes velocity first, then position, and only later tracking error or landing reward. A simplified acceleration-controlled system illustrates this:

$$q_{t+1} = q_t + hv_t, \qquad v_{t+1} = v_t + ha_t,$$

where h is the time step. Holding subsequent actions fixed gives

$$\frac{\partial q_{t+1}}{\partial a_t} = 0, \qquad \frac{\partial q_{t+2}}{\partial a_t} = h^2.$$

The action enters velocity before position. If the objective considers only next-step position reward, this effect has not yet reached the evaluated variable; unrolling to later positions lets the chain rule express it. Second-order position dynamics over time are assembled from first-order derivatives across multiple steps.

Contact in a humanoid further changes the path of influence. In flight, internal joint forces can change body configuration and the distribution of rotation, while center-of-mass translation is determined by external forces. A joint action may have zero derivative with respect to current center-of-mass acceleration yet affect the next contact and later return through landing posture. The evaluation horizon and chosen state variables determine which part of the causal chain the current objective can see.

The two routes for bringing long-term consequences back to actions are now clear: PPO aggregates later rewards into an action's statistical value, while BPTT unrolls intermediate states and propagates through local derivatives. Differentiable models can provide finer directional information. The next question is what difficulties arise in model prediction, long-horizon propagation, and contact boundaries.

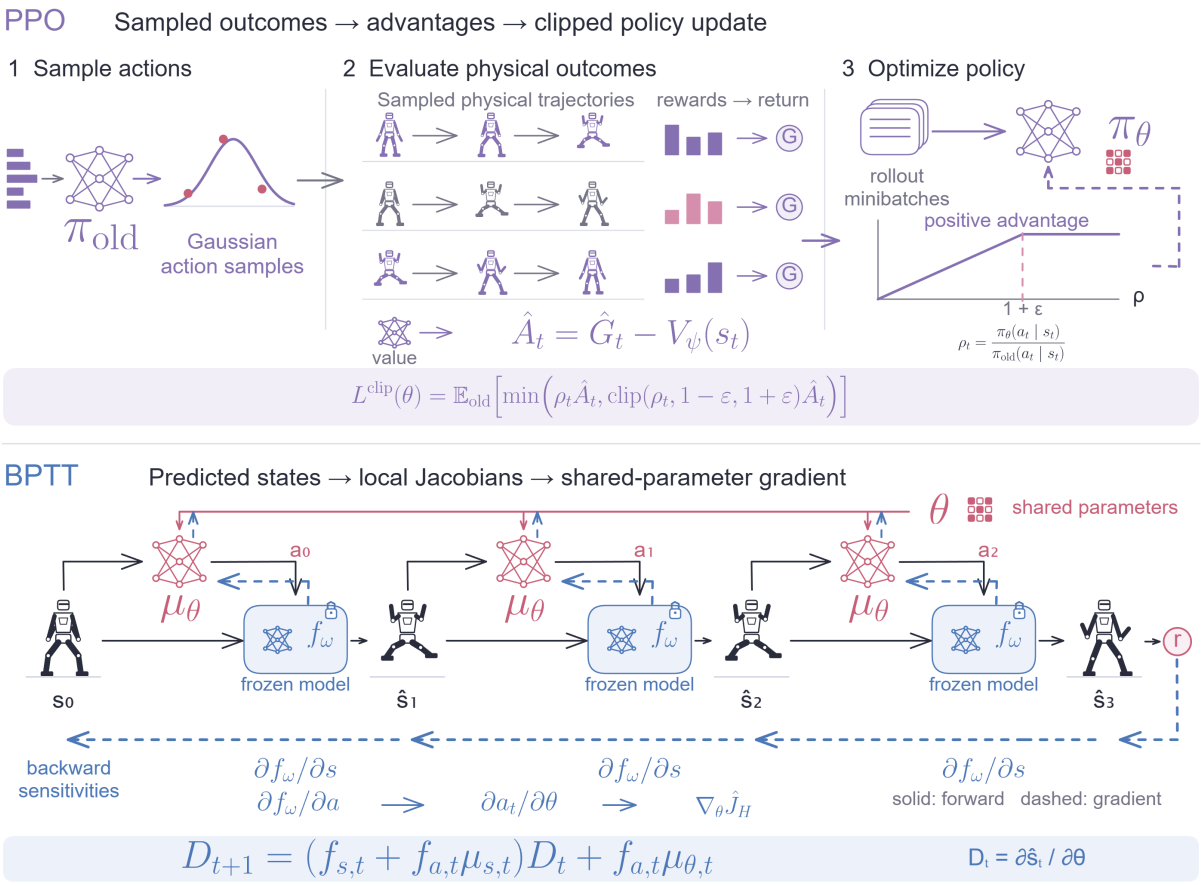


Figure 2. Two credit-assignment paths in motion control. The upper path samples Gaussian-policy actions, subtracts state value from return to obtain advantage, and updates PPO's clipped objective; the piecewise curve shows clipping for positive advantage. The lower path uses a final-step reward and backpropagates

through state and action branches, with policy parameters θ shared across time. Solid lines show forward computation; dashed lines show gradients.

4 Analyzing BPTT

BPTT obtains directions from the model's local changes and propagates them through time to reward. Model-fitting errors, long-horizon Jacobians, and contact boundaries therefore all enter policy updates. Inspecting this computational chain explains why a natural modeling idea needs specific training designs to become useful for control.

4.1 Model Error and Derivative Error

First consider what the model learns. Dynamics fitting is usually supervised learning: the input is $x = (s, a, \xi)$, where ξ records known physical conditions such as friction; the target is next state $Y = s'$, and squared loss trains a point prediction $m(x)$. For a fixed input, write the conditional mean as $\bar{y} = \mathbb{E}[Y \mid x]$. Then

$$\mathbb{E}[\|Y - m\|^2 \mid x] = \mathbb{E}[\|Y - \bar{y}\|^2 \mid x] + \|m - \bar{y}\|^2. \tag{4.1}$$

The cross term is zero, and the first term does not depend on prediction m, so the optimal prediction is

$$m^*(x) = \boxed{\mathbb{E}[Y \mid x]}. \tag{4.2}$$

MSE thus compresses information not determined by the input into a conditional mean. If the same input hides both leftward and rightward slipping outcomes, the mean may fall between them. Treating that intermediate state as the actual future in a rollout can put the next contact and reward in the wrong place. The underlying issue is

$$r(\mathbb{E}[Y \mid x]) \neq \mathbb{E}[r(Y) \mid x].$$

For example, let Y take values -1 and 1 with equal probability, and let $r(y) = y^2$. The reward at the mean state is $r(0) = 0$, while the expected reward under the actual distribution is 1. Outcomes in separate branches are compressed by point prediction into a state that subsequent nonlinear computation cannot handle correctly.

The statistical assumption behind squared loss can also be written directly. If we choose a Gaussian output with fixed covariance,

$$p_\omega(Y \mid x) = \mathcal{N}(f_\omega(x), \sigma^2 I),$$

its negative log-likelihood is

$$-\log p_\omega(Y \mid x) = \frac{\|Y - f_\omega(x)\|^2}{2\sigma^2} + C.$$

MSE therefore corresponds to this Gaussian NLL. Other likelihoods can model continuous distributions; binary termination naturally corresponds to a Bernoulli distribution and BCE. The state representation and conditioning inputs determine how much hidden uncertainty a simple point prediction must absorb.

This makes model inputs crucial. In a deterministic simulator with full state, action, and physical parameters given, the transition can be a single-valued mapping. Providing known friction, calibration offsets, and other conditions reduces the mixture of outcomes caused by hidden conditions. Once this conditional relationship is specified, the model still needs data to fit the actual transition values.

Returning to the previous chapter's BPTT equations, policy updates also use f_s and f_a. Supervised loss evaluates the distance between predicted and true values, whereas these Jacobians describe how neighboring states and actions change a prediction. The two can differ greatly. Consider

$$f(x) = x, \qquad \hat{f}_\epsilon(x) = x + \epsilon \sin(x/\epsilon^2).$$

Its function error satisfies $|\hat{f}_\epsilon - f| \le \epsilon$, but

$$\hat{f}'_\epsilon(x) - f'(x) = \boxed{\epsilon^{-1}\cos(x/\epsilon^2)}. \tag{4.3}$$

A very small amplitude can accompany a very large slope. For BPTT, the model must place trajectories in plausible states and provide nearby directions that guide action improvement.

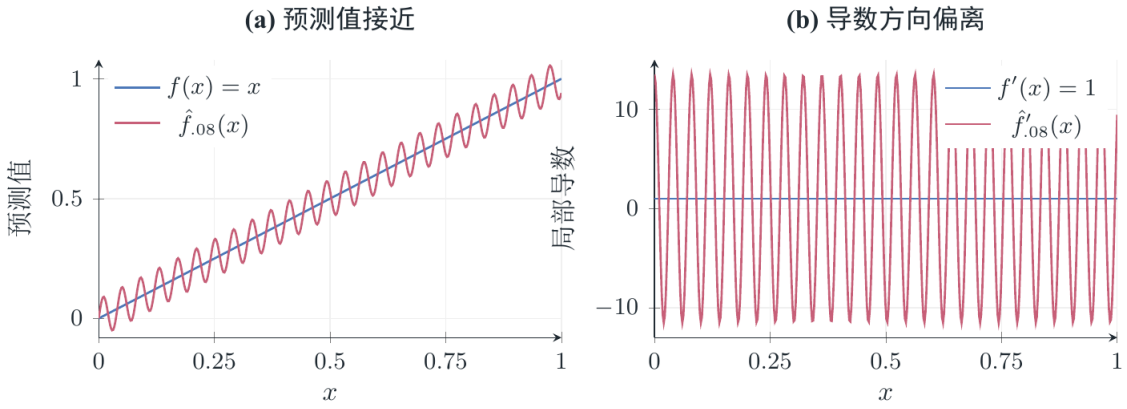


Figure 3. Values and derivatives of the same pair of functions. A prediction-error amplitude of 0.08 corresponds to a derivative-error amplitude of 12.5. State loss supervises values; BPTT uses local directions of change.

These errors occur between the learned model and the simulator that trains it. Real execution also requires bridging the simulator-to-body gap. At the same state and action, let F_{sim} and F_{real} be simulator and real-system transitions, respectively. Then

$$\|f_\omega - F_{\text{real}}\| \le \underbrace{\|f_\omega - F_{\text{sim}}\|}_{\text{surrogate fitting error}} + \underbrace{\|F_{\text{sim}} - F_{\text{real}}\|}_{\text{simulation-to-real dynamics gap}}. \tag{4.4}$$

Supervised fitting directly addresses the first term. Parameter identification and real interaction data correct simulated dynamics, while randomized training helps a policy adapt to a family of physical conditions. ASAP, introduced earlier, addresses this second gap. For differentiable MBRL, these two approximations connect model-based optimization to real-robot deployment.

4.2 Long-Horizon Jacobians and Gradient Propagation

A local model's directions must pass through many later steps to evaluate a takeoff or recovery action. The term $F_{T-1} \cdots F_1$ in Equation (3.4) is exactly this propagation chain. Its norm is bounded by

$$\left\| \frac{\partial \hat{s}_T}{\partial a_0} \right\| \leq \left(\prod_{t=1}^{T-1} \|F_t\| \right) \|f_{a,0}\|.$$

Repeated contraction along a direction makes distant signals weak by the time they reach early actions; repeated stretching also amplifies trajectory and derivative errors. In multiple dimensions, sensitive directions keep changing. Contact makes these local mappings vary rapidly with state, so long rollouts add computational cost, accumulated model error, and gradient instability together.

Take takeoff and landing as an example. The takeoff action can have a large causal effect on eventual success. Computing a gradient must track that influence locally through velocity, posture, contact, and later feedback. If one segment strongly compresses the relevant direction, landing reward struggles to produce a stable takeoff update through dozens of local products. If contact greatly amplifies it, the update direction may become unusually sensitive to tiny changes.

A short rollout eases this burden but raises a value-estimation question: how much future benefit does the current state imply after the window ends? SHAC answers it with a terminal critic:

$$\hat{J}_H = \mathbb{E}\left[\sum_{t=0}^{H-1} \gamma^t \hat{r}_{t+1} + \boxed{\gamma^H V_\psi(\hat{s}_H)} \right]. \tag{4.5}$$

The terminal term contributes the following policy derivative:

$$\gamma^H \underbrace{\frac{\partial V_\psi}{\partial \hat{s}_H}}_{\text{terminal slope of value beyond the window}} \underbrace{\frac{d\hat{s}_H}{d\theta}}_{\text{dynamics propagation inside the window}}. \tag{4.6}$$

The critic compresses later return into a value function of the terminal state, and the BPTT dynamics chain stops at step H. For a robot that has not yet landed, torso orientation, angular velocity, and foot positions can predict future stable return. Once the critic learns these relationships, its terminal-state value slope can guide actions inside the window. [15]

4.3 Contact Thresholds and Gradient Estimation

Contact switches and termination introduce another local problem: changing actions can move more samples across a success boundary even though individual samples have zero reward slope on either side. A one-step task makes this missing gradient information clear. Consider a Gaussian action and a threshold success reward:

$$a = \theta + \sigma z, \qquad z \sim \mathcal{N}(0,1), \qquad R(a) = \mathbf{1}\{a > b\}.$$

Computing the expected objective from the success probability first gives

$$J(\theta) = \Phi\left(\frac{\theta - b}{\sigma}\right), \qquad J'(\theta) = \boxed{\frac{1}{\sigma}\phi\left(\frac{\theta - b}{\sigma}\right)} > 0, \tag{4.7}$$

where Φ, ϕ are the standard normal CDF and density. Moving the parameter rightward shifts more probability mass across the success boundary and increases success probability.

For ordinary pathwise differentiation of individual samples, the threshold function has zero derivative almost everywhere:

$$\mathbb{E}_z\left[\frac{dR(\theta+\sigma z)}{d\theta}\right] = 0. \tag{4.8}$$

Differentiating the action density instead gives the score-function expression

$$\mathbb{E}_a\left[R(a)\frac{a-\theta}{\sigma^2}\right] = \frac{1}{\sigma}\phi\left(\frac{\theta-b}{\sigma}\right). \tag{4.9}$$

The difference comes from the boundary contribution at the hard threshold. Moving the parameter changes the probability mass crossing that boundary, but this change disappears in ordinary sample-wise derivatives. Thus, even a model that predicts every action's reward accurately can miss the direction of change in the expected objective when directly differentiated. Suh and colleagues connect discontinuous boundaries, steep contacts, and gradient-estimation quality. [16]

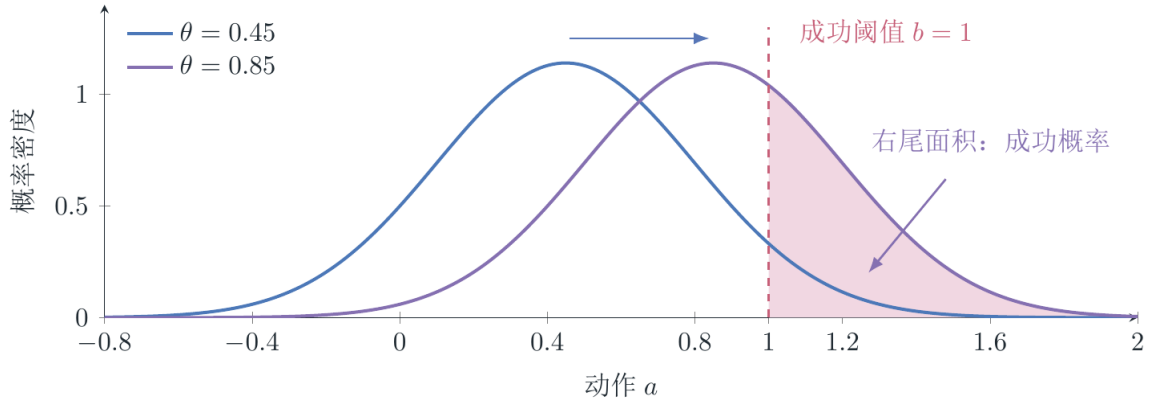


Figure 4. Shifting the action distribution rightward increases the probability mass beyond the threshold. Individual rewards have zero local slope, while the distribution-level success probability still has a clear direction of change. Here, $\sigma = 0.35$.

This difference can change the training direction. Add an action cost and let $R(a) = \mathbf{1}\{a > b\} - \beta a^2$. Then

$$J'(\theta) = \frac{1}{\sigma}\phi\left(\frac{\theta-b}{\sigma}\right) - 2\beta\theta, \qquad \mathbb{E}[g_{\text{path}}] = -2\beta\theta.$$

Take $\beta > 0$ and $\theta > 0$. When the benefit from increased success probability exceeds the cost, the true objective favors a larger action; the gradient missing the boundary contribution keeps reducing it. Training may settle on conserving action without crossing the success boundary.

A continuous but steep contact transition creates a different sampling difficulty. Let $R_\delta(a) = h((a-b)/\delta)$; then $R'_\delta(a) = \delta^{-1}h'((a-b)/\delta)$. As the transition region narrows, fewer samples enter it, while derivatives inside grow larger. A finite batch may repeatedly see only flat regions and only occasionally encounter large gradients that carry much of the expectation. Suh connects this distortion of typical finite-sample estimates to stiffness; AHAC adaptively adjusts the rollout horizon to reduce the burden of propagating gradients through steep contact segments. [16], [17]

Long products and contact boundaries correspond to two improvements in prior work. SHAC uses terminal value to represent distant return and shorten the dynamics chain. Suh and colleagues mix pathwise and score gradients according to estimator quality, in the form $g_\alpha = \alpha g_{\text{path}} + (1-\alpha) g_{\text{score}}$, changing the source of gradient information. SuperMimic's short-horizon actor–critic uses the former idea.

4.4 Data Coverage and Exploration

The discussion so far starts from an existing trajectory and examines the gradient along it. Training also needs to answer an earlier question: has the robot tried useful behaviors, and has the model seen the corresponding transitions? The data distribution directly determines the model's supervision:

$$\mathcal{L}_{\text{WM}}(\omega) = \mathbb{E}_{(s,a,s')\sim\mathcal{B}}\big[\ell(f_\omega(s,a), s')\big]. \tag{4.10}$$

Replay buffer $\mathcal{B}$ determines which transitions the model sees. If standing and normal walking are common but single-foot slips, unbalanced landings, and recovery steps are rare, those regions lack supervision. Optimizing the policy in the model then changes the next collection distribution. Through this loop, model and policy jointly shape the range of learnable behaviors.

Local model-based optimization determines how the policy moves from its current behavior to something better. Local gradients describe changes near the present trajectory. A large step might restore balance, but a small action adjustment may initially raise energy cost or tracking error, making that new path hard to reach by local updates. SuperDyno still identifies discovering anti-fall stepping as difficult. [10] Model accuracy, behavior coverage, and local optimization interact here.

Data collection is therefore part of model-based training. Random actions broaden the action neighborhood seen by the model; reference-state resets cover different motion phases; difficult-segment sampling increases practice in weak regions; and the critic feeds later outcomes back to the current state. SuperMimic's training components have now appeared: continually add transitions, fit a useful short-horizon model, connect long-term consequences with value, and return the policy to the simulator for evaluation and collection.

The discussion above gives three main properties of differentiable MBRL:

1. **Dynamics reuse and sample efficiency.** The same body dynamics supports multiple motions and policy updates; model derivatives directly provide action-improvement directions, using interaction data and imagined rollouts more efficiently.
2. **Data coverage and exploration.** Insufficient exploration leaves important states and actions without model supervision, and local model-based gradients struggle to discover new actions far from current behavior.
3. **Gradient quality and estimation bias.** Model and derivative errors affect update directions; long products can produce vanishing or exploding gradients; at hard thresholds, sample-wise differentiation can miss changes in probability mass crossing a boundary and yield biased estimates.

5 SuperMimic

The derivations above show how differentiable dynamics provides action-optimization directions through model derivatives. Training coverage, prediction error, and gradient-propagation length all affect these directions. SuperMimic organizes training around these issues: the policy continually returns to the simulator

for data, the model receives multi-step prediction supervision over short windows, the actor improves through a short dynamics chain, and the critic handles return beyond the window.

5.1 Alternating Training and Experience Replay

During training, the policy progresses from standing to running and jumping, and also visits unbalanced and landing states. The model needs transitions from these new regions to keep predicting useful action consequences. SuperMimic alternates improvements to policy, world model, and critic in one loop, with parameters θ, ω, ψ, respectively. Physical state s_t produces proprioceptive observation o_t. Goal g_t comes from reference motion by default but can use other goal interfaces. The deterministic actor outputs normalized joint setpoints from observations and goals:

$$a_t = \pi_\theta(o_t, g_t).$$

The actor determines which states the next collection round visits, the model learns action consequences there, and the critic estimates the long-term return from continued control. Figure 5 connects them through this data loop. [1] (§3)

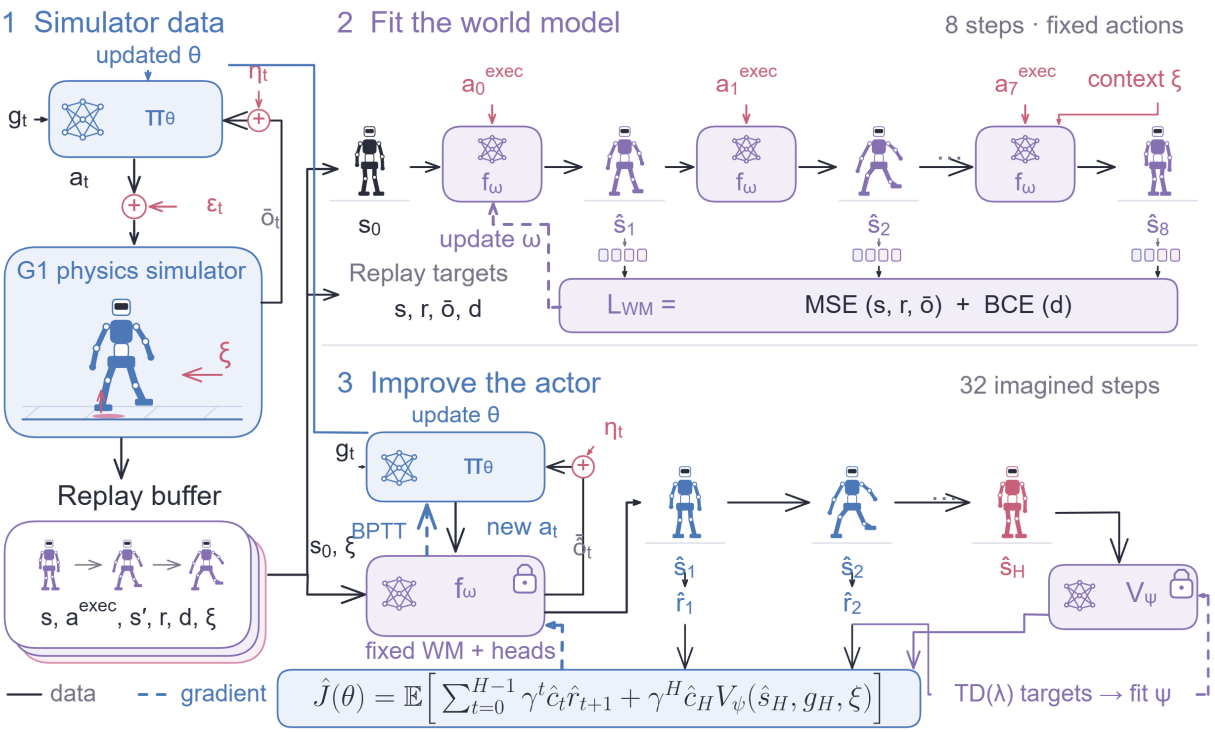


Figure 5. Alternating training in SuperMimic. Model fitting unrolls logged actions, while policy improvement unrolls new actions from the current actor; both start from real simulator states in replay. Solid lines carry data and dashed lines indicate parameter updates: fitting updates ω; the actor updates θ through a frozen world model and terminal value; the critic fits ψ using TD(λ) targets.

Each iteration first collects transitions with the current actor in the physics simulator. Execution can add noise $a_t^{\text{exec}} = a_t + \epsilon_t$, and replay records the actual executed action a_t^{exec} and its consequences for model fitting. Imagined rollouts then start from recorded states: the current actor generates new actions in the model, receives rewards, and is optimized through the frozen model while the critic is fitted. The updated policy returns to the simulator and adds the states and actions it now encounters.

Replay also retains older experience. Given state, executed action, and physical conditions, past and current policies obey the same transition law, so old data remains useful for dynamics supervision. New data brings fitting into regions the policy is expanding into. This Dyna-style alternation puts model coverage and policy improvement in a single feedback loop. [18]

5.2 World Models and Multi-Step Supervision

Once data enters the buffer, the model must learn to keep using its own predictions. A one-step error changes the input at the second step and then affects later contact and posture, so training must expose this accumulation. To support full control rollouts, the world model provides four interfaces: dynamics, reward, observation, and termination:

$$\begin{aligned} \hat{s}_{t+1} &= f_\omega(\hat{s}_t, a_t, \xi), \quad \hat{r}_{t+1} = r_\omega(\hat{s}_t, a_t, g_t), \\ \hat{\bar{o}}_t &= g_\omega(\hat{s}_t, \xi), \qquad\quad \hat{d}_t = d_\omega(\hat{s}_t, g_t). \end{aligned}$$

Here, ξ is episode-level physical context, including friction, joint calibration offsets, and torso center-of-mass offsets. The same posture and action can have different consequences under different friction. Feeding known ξ into the model makes these transitions conditionally learnable. Heads that can be computed analytically from state are computed directly; the others are learned.

Consider replay window $(s_0, a_0, s_1, \ldots, a_7, s_8)$. Starting at $\hat{s}_0 = s_0$, the model predicts all subsequent states itself:

$$\hat{s}_1 = f_\omega(s_0, a_0, \xi), \quad \hat{s}_2 = f_\omega(\hat{s}_1, a_1, \xi), \quad \ldots, \quad \hat{s}_8 = f_\omega(\hat{s}_7, a_7, \xi).$$

Actions are fixed to the log, while states evolve autoregressively. Later errors therefore also train earlier predictions, teaching the model to continue from its own generated inputs. The fitting objective is

$$\begin{aligned} \mathcal{L}_{\text{WM}}(\omega) = \mathbb{E}_{\mathcal{B}} \sum_{j=1}^{8} \Big[&\|\hat{s}_j - s_j\|^2 + (\hat{r}_j - r_j)^2 \\ &+ \|\hat{\bar{o}}_j - \bar{o}_j\|^2 + \text{BCE}(\hat{d}_j, d_j) \Big]. \end{aligned} \tag{5.1}$$

Loss terms for analytical heads are omitted. Continuous states, rewards, and noise-free observations use regression; termination labels use binary cross-entropy. Here, $k = 8$ specifies the supervised log window, while the later $H = 32$ specifies how far the actor uses the model to obtain gradients. The former exposes multi-step prediction error; the latter determines the length of dynamics traversed by one policy update.

Sensor noise serves another purpose: preparing the actor for disturbances in actual feedback. Its sampling process is known in the simulator, so it can be separated from dynamics fitting:

$$o_t = \bar{o}_t + \eta_t, \qquad \hat{o}_t = \hat{\bar{o}}_t + \eta_t. \tag{5.2}$$

The model first predicts a noise-free observation, then sensor noise is sampled externally and added before the actor receives it. Physical variation thus enters transition conditioning through ξ, while sensor variation enters the controller through η_t. The model focuses on body transitions under given conditions, and the actor learns continuous control from noisy feedback.

5.3 Short Rollouts and Terminal Value

Even with a multi-step model, policy optimization faces products of long-horizon Jacobians. SuperMimic explicitly unrolls only a short trajectory per update and uses a terminal critic to continue future return. Rollouts start from replay states and their corresponding ξ, receiving predicted observations with added noise at each step. Model parameters ω stay fixed during the actor update, while state and action Jacobians remain in backpropagation.

A short window can still cross a failure boundary. Cumulative survival weights first express the removal of rewards after termination. In simulator logs, $d_j \in \{0, 1\}$, and cumulative survival weight is defined as

$$c_0 = 1, \qquad c_{t+1} = c_t(1 - d_{t+1}), \qquad c_t = \prod_{j=1}^{t}(1 - d_j). \tag{5.3}$$

In model rollouts, binary d_j is replaced by predicted probability $\hat{d}_j \in [0, 1]$ in the same recurrence, giving continuous survival weight $\hat{c}_t$. Termination risk can then enter imagined return and its gradient as state changes. The actual simulator objective is $J = \mathbb{E}\sum_{t\geq 0}\gamma^t c_t r_{t+1}$; the model-based actor objective is

$$\hat{J}(\theta) = \mathbb{E}\Big[\boxed{\sum_{t=0}^{H-1}\gamma^t \hat{c}_t \hat{r}_{t+1}} + \boxed{\gamma^H \hat{c}_H V_\psi(\hat{s}_H, g_H, \xi)}\Big], \tag{5.4}$$

The default is $H = 32$ at 50 Hz, corresponding to 0.64 seconds. The blue term computes rewards through dynamics inside the window, while the purple term uses V_ψ to predict cumulative return beyond it. The actor can therefore learn more distant consequences through terminal value while limiting explicit dynamics backpropagation to 32 steps. Survival weights account for termination risk in both terms.

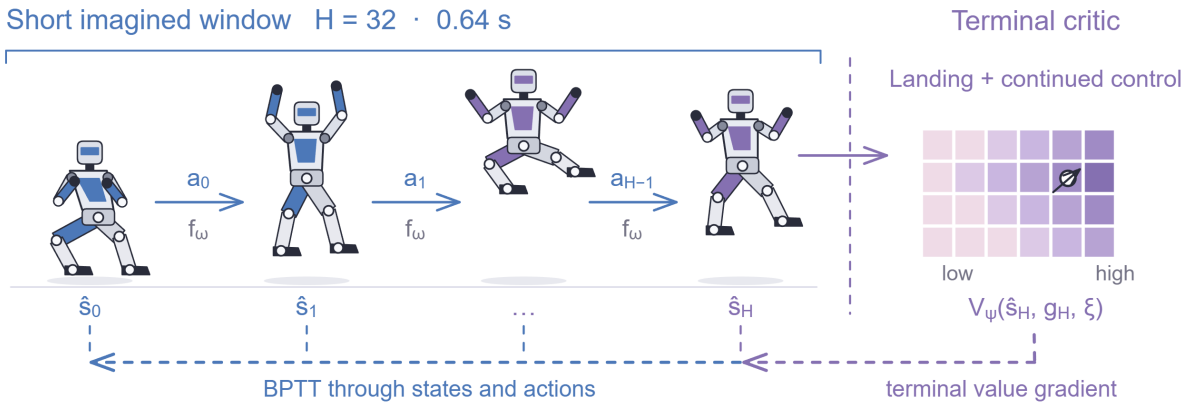


Figure 6. The critic compresses more distant futures into the window's endpoint. The actor uses $\partial V_\psi / \partial s_H$ and updates actions through the model chain inside the window.

For a takeoff action, the robot may still be airborne when the window ends. Terminal posture and angular velocity already contain clues about landing; the critic maps them to later return, forming the value gradient that flows back into the window. Reference motion describes the desired target, while the critic evaluates continued control from the actual current state. A posture that is briefly close to the reference but hard to recover from later will also affect value through subsequent return.

5.4 Termination Risk and Survival Weights

The previous threshold example showed that ordinary sample-wise differentiation at a hard threshold misses probability mass crossing the boundary. In SuperMimic's imagined objective, a continuous termination head changes how much future return remains available according to predicted termination probability. Define the within-window recursive return as

$$U_H = V_\psi(\hat{s}_H, g_H, \xi), \qquad U_t = \hat{r}_{t+1} + \gamma(1 - \hat{d}_{t+1})U_{t+1}.$$

Expanding U_0 yields the single-trajectory objective in Equation (5.4). Differentiation produces three terms:

$$\frac{dU_t}{d\theta} = \frac{d\hat{r}_{t+1}}{d\theta} + \gamma(1 - \hat{d}_{t+1})\frac{dU_{t+1}}{d\theta} - \boxed{\gamma U_{t+1}\frac{d\hat{d}_{t+1}}{d\theta}}. \tag{5.5}$$

The last term measures the effect of a termination-probability change, weighted by future value U_{t+1}: when future value is positive, increasing termination probability lowers the objective. The continuous head thus incorporates lost future benefit near the failure boundary into the model gradient. The reward on the transition into termination is still multiplied by the previous c_t; only later rewards are switched off by subsequent weights.

The termination head provides derivatives of continuation risk, while learned dynamics provides derivatives of contact and motion consequences. Together with per-step tracking reward, they give the actor update directions from posture error and future benefit before it finishes a full motion. These surrogates are continually fitted from simulator data and used for policy optimization over short windows.

5.5 Critics and Multi-Step Returns

How terminal value learns the benefit of landing and recovery depends on its training targets. SuperMimic computes multi-step returns backward from imagined transitions, combining within-window rewards with more distant value estimates. Let $V_t = V_\psi(\hat{s}_t, g_t, \xi)$ and $m_{t+1} = 1 - \hat{d}_{t+1}$. The paper uses the λ-return:

$$\hat{R}_t = \hat{r}_{t+1} + \gamma m_{t+1}\left[(1-\lambda)V_{t+1} + \lambda\hat{R}_{t+1}\right], \quad \hat{R}_H = V_H. \tag{5.6}$$

With $\lambda = 0$, next-state value gives a one-step bootstrap. With $\lambda = 1$, rewards accumulate to the end of the window and continue through V_H. Intermediate values mix targets of different lengths. The defaults are $\gamma = 0.99, \lambda = 0.95$.

These multi-step returns also connect to generalized advantage estimation (GAE), commonly used in PPO. Let $\delta_t = \hat{r}_{t+1} + \gamma m_{t+1}V_{t+1} - V_t$. Then

$$\hat{R}_t - V_t = \delta_t + \gamma\lambda m_{t+1}(\hat{R}_{t+1} - V_{t+1}).$$

Unrolling further gives

$$\hat{R}_t - V_t = \sum_{l=0}^{H-t-1}(\gamma\lambda)^l\left(\prod_{i=1}^{l} m_{t+i}\right)\delta_{t+l}. \tag{5.7}$$

The temporally and survival-weighted TD errors on the right give $\hat{R}_t - V_t$, a multi-step advantage estimate. Adding back V_t gives return target $\hat{R}_t$ for the critic. SuperMimic fits

$$\mathcal{L}_V(\psi) = \mathbb{E} \sum_{t=0}^{H-1} \left[V_\psi(\hat{s}_t, g_t, \xi) - \mathrm{sg}(\hat{R}_t) \right]^2 . \tag{5.8}$$

Here, sg denotes a target held fixed during this fitting step. Targets are computed once per rollout and used to update critic parameters. During the actor update, ψ is fixed, but the derivative of value with respect to terminal state remains. Multi-step returns bring later rewards into value targets; BPTT then improves within-window actions through the resulting value slope. [13], [15], [1]

5.6 Data Sampling and Deployment Interfaces

Both model and value learn from visited states, bringing us back to coverage. Reference motion provides the timing of takeoff, rotation, and landing; dense reward measures tracking error at each step; and training sampling allocates practice across phases. SuperMimic divides motion timelines into one-second bins and maintains return G_b for each bin. The current difficulty distribution is

$$q_{\text{hard}}(b) \propto \exp\left(-25 \frac{G_b}{G_{\max}} \right) .$$

Resets mix 50% uniform sampling, 40% current difficulty, and 10% historical difficulty. If a flip can take off but often fails before landing, its lower local return increases practice near that phase. The uniform component covers the full library; the historical component retains practice on previously difficult segments. Model imagination samples replay starting points and uses these actually visited simulator states for short-window optimization.

Timeline sampling expands phase coverage, while collection-time action noise expands action coverage near the same state, with default $\epsilon_t \sim \mathcal{N}(0, 0.1^2 I)$. These executed noisy actions supervise the model; the actor backpropagates through imagined trajectories from its current deterministic policy.

The data-and-model loop must also connect to inputs available on real hardware. The model and critic can use full state and ξ in training, while the actor receives the last ten proprioceptive frames and the next ten reference frames. Physical randomization covers friction, joint offsets, and center-of-mass offsets; sensor noise covers joint positions and velocities, IMU angular velocity, and projected gravity. Physical variation exposes the policy to different body and contact parameters, while sensor variation prepares it for feedback disturbances. Privileged training information remains on the model and critic side.

Policy outputs become joint setpoints $q^{\text{sp}} = \alpha a + q^0$ and are executed by low-level PD control. The real-robot control path runs the actor and joint controller, while the world model and critic handle training computation. The paper uses flat-ground simulation and removes sudden root-velocity pushes during training: state jumps not recorded as inputs disrupt deterministic transition fitting. Push recovery is evaluated separately afterward.

Each difficulty from the previous chapter now has a concrete treatment: replay and multi-step fitting track model error; short rollouts and the critic shorten dynamics-gradient chains; continuous survival weights express termination risk; and references, resets, and action noise change coverage. Experiments must now show whether these choices reduce training compute, whether the critic changes behavior beyond the window, and whether the same model stays useful as the motion library grows.

6 Results

6.1 Training Efficiency and Motion Scale

On the large motion library, SuperMimic uses 264.56 hours of public motion data and exceeds the released SONIC checkpoint's normalized return after three hours of the complete training loop on one H100. SONIC reports 611 hours of data, 128 L40 GPUs, and 168 hours of training; the paper's thousand-fold GPU-hour result refers to this system comparison. After 9.8 hours on two H100s, SuperMimic achieves zero failures over 129,514 training motions evaluated under a shared termination condition. [1] (§4.1, App. F) Normalized return divides undiscounted reward sum by the maximum reward sum for the complete reference clip; early termination lowers it.

On the same H100, model rollout throughput is more than ten times MuJoCo Warp's across the 2k–16k parallelism range used in practice. Inside the same learned world model, analytic updates reach PPO-WM's best training return with about $1/64$ of the imagined steps (Figure 7). As the motion library grows from 1 hour to approximately 264 hours, normalized eight-step state prediction error stays in the 0.007–0.011 range. [1] (§4.2, Fig. 3)

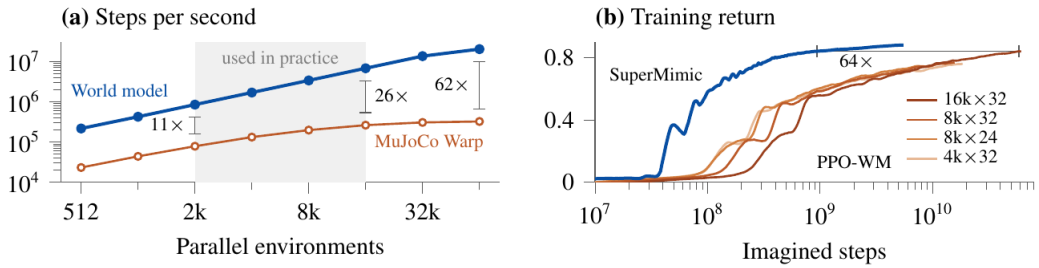


Figure 7. Figure 5 from the paper. Left: forward throughput of model and simulator on an H100. Right: walk/run/sprint training inside the model, comparing analytic updates with PPO-WM at different batch sizes.

6.2 Horizon Ablations and Push Recovery

Figure 8 shows a PPO-Sim comparison and horizon ablations under matched task settings. SuperMimic reaches evaluation return 0.82 in approximately 1.2 hours; PPO-Sim does not reach it within an 18-hour budget. At $H = 16$, the version without a critic misses the target; adding a critic matches the critic-free version at $H = 32$. The default $H = 32$ with a critic reaches the target fastest, while longer windows slow training. [1] (§4.3)

After the policies reach similar normal tracking returns, the paper applies velocity perturbations of 0.5–4 m/s to a static T-pose. The critic-trained policy stabilizes within 0.5–2.4 seconds. The version without a critic keeps stumbling after a 0.5 m/s perturbation and falls at 4 m/s. [1] (§4.3)

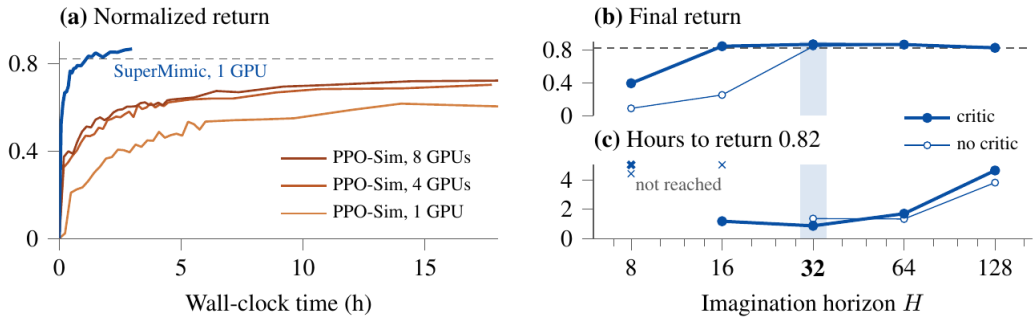


Figure 8. Figure 4 from the paper. Left: PPO-Sim on LAFAN1 with the same motions, observations, rewards, randomization, and network size. Right: critic and rollout-length ablations; crosses indicate failure to reach the target.

6.3 Real G1 Control

The same trained tracker performs hopping, side rolls, 90° flips, and in-place 360° flips on a real G1 without per-motion tuning. Deployment runs the actor and low-level joint controller; the world model and critic remain in training. [1] (§4.4)



Figure 9. The complete motion sequences from the paper's Figure 6: hopping, side roll, 90° flip, and in-place 360° flip, from top to bottom.

7 Reproduction

This reproduction uses the third-party Uni-SuperMimic implementation to train a G1 simulation tracking policy on one H100 80GB. Its LAFAN1 library contains 40 motions, approximately 2.45 hours of reference data. Training uses 4096 parallel environments for 12000 iterations, taking approximately **4 hours 19 minutes**. After training, each reference motion is evaluated in full from frame 0 with observation and action noise disabled. **All 40/40 motions complete**, with a mean normalized full-clip return of **0.833**.

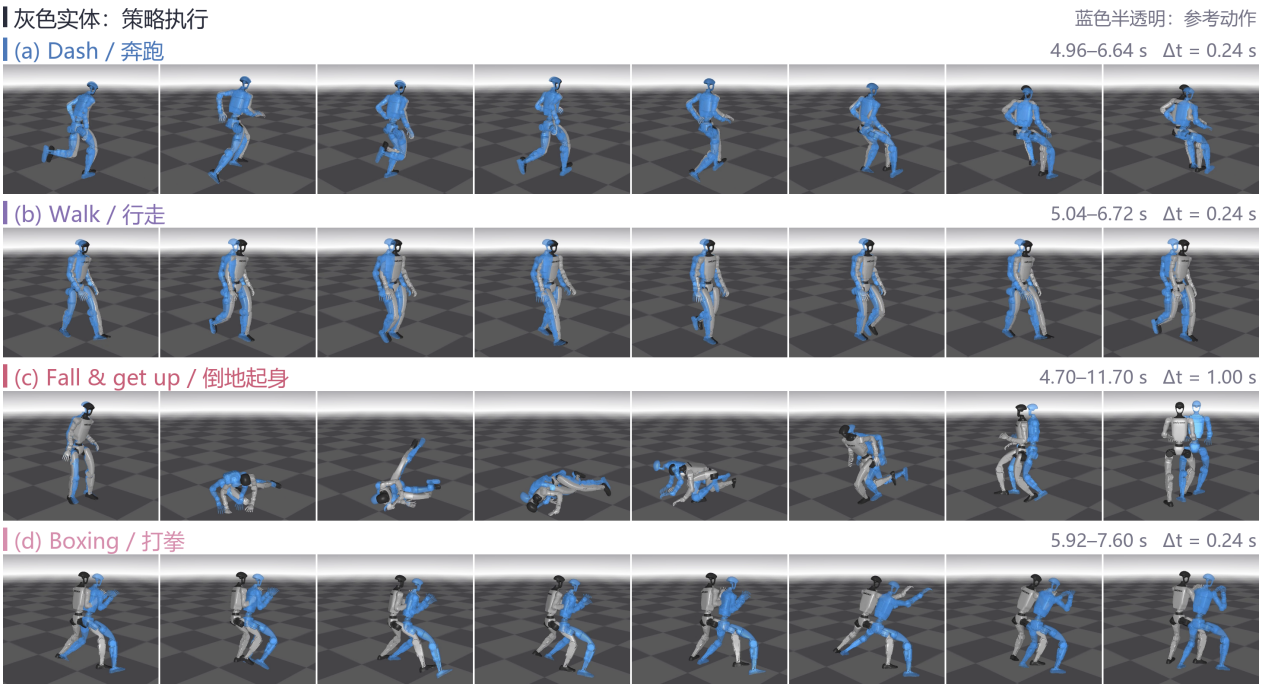


Figure 10. MuJoCo replays of the reproduced policy. Each row contains eight frames: running, walking, falling and getting up, and punching, from top to bottom. Solid gray shows policy execution; translucent blue shows the reference.

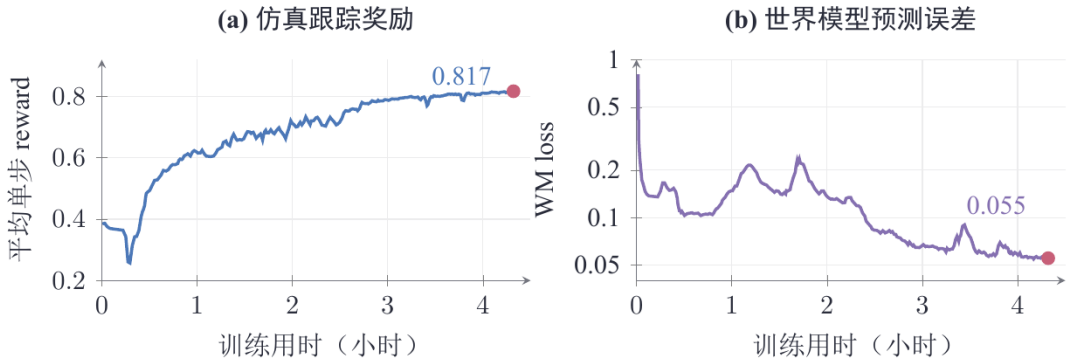


Figure 11. Training curves from this reproduction. Left: mean per-step tracking reward in physics simulation. Right: multi-step world-model prediction loss on a logarithmic axis. Data comes from the original training log, recorded every 50 iterations.

8 Action World and WAM

SuperMimic learns action-conditioned body dynamics well and repeatedly uses it to train controllers. The world here mainly consists of posture, velocity, and contact changes of the same humanoid body on flat ground. Adding motions mainly expands the visited range of these dynamics. [1] (§6, App. E)

As World Action Models (WAMs) move toward object manipulation, action consequences also vary with objects and scenes: pulling a drawer requires considering its rails; picking up a cup involves support and tipping; vision may obscure contact. We look forward to seeing how WAMs learn reusable action consequences under more complex objects, constraints, and scenes, and use those predictions to improve control.

A Notation and Training Interfaces

Symbol	Meaning
$s_t, \hat{s}_t$	Simulator physical state and model-predicted state.
$o_t, \bar{o}_t$	Actor observation and noise-free observation; $o_t = \bar{o}_t + \eta_t$.
g_t, a_t	Goal conditioning and normalized joint setpoints.
r_{t+1}, d_{t+1}	Reward after action execution and termination label at the reached state.
θ, ω, ψ	Learnable parameters of actor, world model, and critic.
ξ, η_t, ϵ_t	Episode physical context, observation noise, and collection-time action noise.
D_t, F_t, B_t	Total state derivative with respect to policy parameters, closed-loop state Jacobian, and current-step parameter injection.
$c_t, \hat{c}_t$	Cumulative survival weights in actual and imagined trajectories.
k, H, γ, λ	Model-fitting window, policy-imagination window, discount, and return-mixture coefficient.

Table 1. Main notation used in these notes.

Item	SuperMimic default
Model supervision / policy horizon	$k = 8$ / $H = 32$, control time step 20 ms
Discount / λ-return	$\gamma = 0.99$, $\lambda = 0.95$
Model network	MLP, three layers of 2048 units
Actor observations / motion conditioning	Last ten proprioceptive frames / next ten reference frames
Policy batch	4096 parallel imagined rollouts
Model fitting per iteration	Ten batches, each with 2048 replay windows
Critic fitting	Five epochs, four minibatches each
Replay capacity	Last 1000 control steps per environment
Actor gradient handling	Global norm clipped to 1.0, then elementwise clipping to $[-0.1, 0.1]$
Collection-time action noise	$\mathcal{N}(0, 0.1^2 I)$

Table 2. Settings from the paper's Appendices A and D. Actor gradient clipping acts on BPTT derivatives; PPO clipping acts on the probability-ratio benefit in Equation (2.8).

Category	Variable	Uniform sampling range
Physical context ξ	Friction coefficient	$[0.3, 1.2]$
	Joint calibration offset	± 0.01 rad
	Torso center-of-mass offset	$x : \pm 0.025$ m; $y, z : \pm 0.05$ m

Category	Variable	Uniform sampling range
Observation noise η_t	Joint angle / velocity	± 0.01 rad / ± 0.5 rad/s
	IMU angular velocity	± 0.2 rad/s
	Projected gravity	± 0.05 per dimension

Table 3. Specific model conditions and actor-input perturbations. Physical context is sampled per episode; observation noise is added outside the world model.

References

[1] Qiayuan Liao, Yiyang Shao, Junfeng Long, et al. SuperMimic: Scaling Humanoid Whole-Body Control with One Differentiable World Model. CoRL, 2026. Project: `https://supermimic.github.io/`.

[2] Xue Bin Peng, Pieter Abbeel, Sergey Levine, Michiel van de Panne. DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills. ACM TOG, 2018.

[3] Nikita Rudin, David Hoeller, Philipp Reist, Marco Hutter. Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning. CoRL 2021; PMLR 164, 2022.

[4] Tairan He, Zhengyi Luo, Wenli Xiao, et al. Learning Human-to-Humanoid Real-Time Whole-Body Teleoperation. IROS, 2024.

[5] Tairan He, Jiawei Gao, Wenli Xiao, et al. ASAP: Aligning Simulation and Real-World Physics for Learning Agile Humanoid Whole-Body Skills. RSS, 2025.

[6] Yitang Li, Yuanhang Zhang, Wenli Xiao, et al. Hold My Beer: Learning Gentle Humanoid Locomotion and End-Effector Stabilization Control. 2025.

[7] Qiayuan Liao, Takara E. Truong, Xiaoyu Huang, Yuman Gao, Guy Tevet, Koushil Sreenath, C. Karen Liu. BeyondMimic: From Motion Tracking to Versatile Humanoid Control via Guided Diffusion. arXiv:2508.08241v4, 2025.

[8] Zhengyi Luo, Ye Yuan, Tingwu Wang, et al. SONIC: Supersizing Motion Tracking for Natural Humanoid Whole-Body Control. Science Robotics 11(117):eaed4592, 2026; arXiv:2511.07820v4.

[9] Levi Fussell, Kevin Bergamin, Daniel Holden. SuperTrack: Motion Tracking for Physically Simulated Characters using Supervised Learning. ACM TOG 40(6), 2021.

[10] Yu Lei, Zhengyi Luo, Tairan He, Jinkun Cao, Guanya Shi, Kris Kitani. Scalable Humanoid Whole-Body Control via Differentiable Neural Network Dynamics. ICLR Workshop on World Models, 2025.

[11] John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, Pieter Abbeel. Trust Region Policy Optimization. ICML, 2015.

[12] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov. Proximal Policy Optimization Algorithms. 2017.

[13] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, Pieter Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation. ICLR, 2016.

[14] Nicolas Heess, Gregory Wayne, David Silver, Timothy Lillicrap, Tom Erez, Yuval Tassa. Learning Continuous Control Policies by Stochastic Value Gradients. NeurIPS, 2015.

[15] Jie Xu, Viktor Makoviychuk, Yashraj Narang, Fabio Ramos, Wojciech Matusik, Animesh Garg, Miles Macklin. Accelerated Policy Learning with Parallel Differentiable Simulation. ICLR, 2022.

[16] Hyung Ju Suh, Max Simchowitz, Kaiqing Zhang, Russ Tedrake. Do Differentiable Simulators Give Better Policy Gradients? ICML, PMLR 162:20668–20696, 2022.

[17] Ignat Georgiev, Krishnan Srinivasan, Jie Xu, Eric Heiden, Animesh Garg. Adaptive Horizon Actor-Critic for Policy Learning in Contact-Rich Differentiable Simulation. 2024.

[18] Richard S. Sutton. Dyna, an Integrated Architecture for Learning, Planning, and Reacting. SIGART Bulletin 2(4):160–163, 1991.